

# Karate-Ashi v2.1 — Referência Técnica do Core

*Sistema de avaliação de exames de Karatê (OMR + motor de regras + relatórios HTML)*

01 de outubro de 2026

## 1. Visão Geral da Arquitetura do Core

O núcleo computacional do Karate-Ashi v2.1 opera sob uma arquitetura desacoplada e orientada a processamento em lote (batch pipeline). A finalidade primária do motor consiste em converter leituras ópticas de formulários físicos de avaliação de Karatê em relatórios executivos de desempenho, respeitando matrizes pedagógicas estritas e travas éticas de arbitragem.

O ciclo de vida operacional inicia-se na captura digital de folhas impressas com balões posicionalmente mapeados e marcadores QR bidimensionais através de scanner flatbed (A4 paisagem, resolução de 300 a 400 DPI). A ferramenta `tools/ingest_folhas.py` processa os scans brutos, normaliza a geometria para o sistema de coordenadas de referência e gera um arquivo JSON canônico para cada folha processada, contendo a leitura de presença, frequências por quesito e observações pontuais.

Em seguida, o orquestrador `core/pipeline.py` ingere os arquivos intermediários e os submete ao motor de cálculo `core/engine.py`. Neste estágio, a função de entrada `processa_aluno` agrega os dados dos múltiplos avaliadores oficiais (consenso tripartite), calcula as notas ponderadas para cada um dos quatro quesitos e classifica o status normativo do candidato. Simultaneamente, o módulo `core/observacoes_automaticas.py` avalia a distribuição vetorial dos erros cometidos pelo atleta, inferindo diagnósticos pedagógicos automáticos a partir de regras formais.

Finalizado o processamento numérico, o módulo `core/relatorio_html.py` sintetiza as estruturas em dois artefatos independentes: o Relatório do Sensei (visão departamental do dojo com detalhamento individual e próxima graduação) e o Relatório Master (visão macroscópica consolidada de todos os dojos, incluindo rankings, médias por quesito e recomendações de treino). O pipeline conclui o ciclo organizando os arquivos em uma árvore de staging em disco e disparando a sincronização remota via rclone para o repositório em nuvem.

## 2. Módulo de Configuração Central ( `core/config.py` )

O módulo `core/config.py` atua como a fonte única da verdade (Single Source of Truth - SSOT) para tipagens de quesitos, graus de suporte a faixas e carregamento

idempotente de arquivos de configuração serializados em JSON com codificação UTF-8.

## 2.1 Constantes Exportadas

- `QUESITOS = ["kihon", "kata", "bunkai", "kumite"]` : Lista ordenada canônica e imutável dos quatro quesitos que compõem a avaliação oficial. Todas as iterações e matrizes de notas no sistema devem respeitar essa sequência estrita.
- `FAIXAS_SUPORTADAS = ["branca", "amarela", "laranja", "verde", "azul"]` : Identificadores padronizados das graduações que possuem matrizes técnicas completas e homologadas para cálculo no motor v2.1.
- `FAIXAS_PLACEHOLDER = ["roxa", "marrom", "preta"]` : Graduações reservadas cujos esquemas ainda não possuem matriz paramétrica ativa. Operações contra essas faixas devem ser interceptadas previamente para evitar cálculos corrompidos.

## 2.2 Funções do Módulo

```
carregar_json(caminho: Path) -> dict
```

Lê e desserializa um arquivo JSON a partir do disco utilizando codificação estrita UTF-8. Em caso de ausência física do arquivo apontado, levanta explicitamente `FileNotFoundError` com mensagem descritiva do recurso faltante. Caso o conteúdo não obedeça às especificações de formatação da RFC 8259, intercepta a falha e dispara `ValueError` com detalhes do erro de análise léxica.

```
faixas_suportadas(base_cfg: Path) -> list[str]
```

Deriva dinamicamente a lista de faixas ativas a partir da chave `"suportadas"` presente no arquivo `config/faixas.json`. Na ocorrência de exceções de leitura, ausência do arquivo no diretório `base_cfg` ou integridade comprometida, assume comportamento resiliente de fallback, retornando a constante interna `FAIXAS_SUPORTADAS`.

## 3. Motor de Cálculo e Regras de Negócio ( `core/engine.py` )

O componente `core/engine.py` encapsula toda a lógica matemática, ponderações por matrizes técnicas, travas de integridade ética e o sistema de consenso avaliativo do Karate-Ashi.

### 3.1 Constantes Operacionais

- `NOTA_MAX_QUESITO = 25.0` : Teto absoluto de pontuação distribuído para cada um dos quatro quesitos técnicos. A pontuação máxima acumulada do exame é de 100.0 pontos.

### 3.2 Assinaturas e Especificações de Funções

```
carregar_faixa(base_cfg: Path, faixa: str) -> dict
```

Localiza e carrega a matriz de parâmetros do arquivo `config/faixas/{faixa}.json`. O parâmetro `faixa` passa obrigatoriamente por normalização léxica (conversão para

minúsculas e remoção de espaços em branco externos conforme RL-04). Levanta `ValueError` com diagnóstico formal caso o arquivo inexista ou caso a faixa pertença à lista de faixas não suportadas (placeholder).

```
frequencia_media(marcacoes: list[int]) -> float
```

Calcula a média aritmética simples das frequências observadas para determinado critério a partir das anotações de todos os avaliadores presentes. Os valores inteiros variam na escala de 0 a 7. Retorna `0.0` caso a lista fornecida seja vazia.

```
multiplicador_progressivo(fc: float, faixas: list[dict]) -> float
```

Avalia a frequência combinada `fc` contra intervalos discretos de severidade definidos na configuração. Localiza o primeiro registro onde a condição `fc_min <= fc <= fc_max` seja satisfeita e retorna o multiplicador numérico correspondente. Se o valor estiver fora de qualquer intervalo mapeado, retorna `0.0`.

```
desconto_criterio(fc: float, peso: float, mult: float) -> float
```

Determina o impacto subtrativo unitário de uma falha sobre a pontuação do quesito através da equação:

$$\text{desconto} = fc \times \text{peso} \times \text{mult}$$

O resultado final é truncado e arredondado para exatamente duas casas decimais monetárias.

```
consenso_controle(marcacoes_controle: list[int]) -> bool
```

Executa a verificação da trava ética da bancada de arbitragem. Retorna `True` exclusivamente se todos os avaliadores presentes no exame marcaram ao menos uma ocorrência de falta de controle (valor `>= 1`) para o candidato em questão. Se houver divergência, retorna `False`.

```
nota_quesito(avaliacoes: list[dict], quesito: str, criterios_q: list[dict], regras: dict) -> dict
```

Consolida as notas e penalizações de um único quesito técnico. Itera sobre cada critério mapeado na matriz da faixa, calcula a frequência média entre os examinadores e deduz as pontuações. Aplica o teto máximo permissível em caso de ativação de trava ética por falta de controle e insere o marcador `ALERTA_ETICO` quando a anotação for parcial entre os avaliadores. Retorna um dicionário estruturado com as chaves `nota`, `desconto_total`, `detalhes`, `alerta` e `controle_marcacoes`.

```
classificar_status(nota_final: float, regras: dict) -> str
```

Mapeia o valor escalar da nota final contra os limiares normativos definidos no dicionário `regras["status"]`. Retorna a cadeia identificadora correspondente aos

estados `APROVADO` , `RECUPERACAO` OU `REPROVADO` com base nos limites `aprovado_min` e `recuperacao_min` .

```
_validar_entrada(avaliacoes: list[dict], faixa: str | None) -> None
```

Função interna de proteção de borda. Valida se os blocos de avaliação fornecidos não estão nulos, vazios ou corrompidos estruturalmente, impedindo o avanço do fluxo de execução antes do disparo das rotinas matemáticas.

```
_derivar_faixa(avaliacoes: list[dict]) -> str
```

Inspeciona a coleção de avaliações brutas e extrai o identificador da faixa pretendida/atual a partir do primeiro registro válido encontrado na chave `aluno.faixa_atual` .

```
processa_aluno(avaliacoes: list[dict], base_cfg: Path, faixa: str | None = None) -> dict
```

Orquestra o cálculo completo do desempenho individual de um atleta a partir dos dados de múltiplos examinadores. Implementa as diretrizes do Guard v2.0, assegurando que pontuações máximas padrão jamais sejam outorgadas na ausência injustificada de dados.

Conforme estipulado pela regra RL-03, a nota final composta pela soma dos 4 quesitos deve ser arredondada utilizando o algoritmo Decimal `ROUND_HALF_UP` para exatamente 1 casa decimal antes de qualquer chamada a `classificar_status` . Retorna o contrato completo de saída:

json

```
{ "nota_final": 78.5,<br/>  "status": "APROVADO",<br/>  "status_bruto": "APROVADO",<br/>
```

## 4. Módulo de Observações Automáticas

### ( `core/observacoes_automaticas.py` )

Este módulo analisa padrões de erro recorrentes entre diferentes exercícios e formula diagnósticos pedagógicos de treino sem intervenção manual, transformando marcações estatísticas em direcionamentos técnicos claros.

#### 4.1 Regras de Dedução Diagnóstica

1. Regra Transversal: Identifica deficiências técnicas sistêmicas cuja frequência ultrapassa o limiar em 2 ou mais quesitos distintos (ex.: perda de equilíbrio ocorrendo simultaneamente em Kihon e Kata).
2. Regra Concentrada: Detecta falhas graves isoladas cuja intensidade atinge a frequência máxima observável em um único quesito especializado.
3. Regra de Ranking: Destaca o critério que acumulou o maior somatório bruto de penalidades ao longo de todo o exame.

4. Regra Percentual: Sinaliza falhas repetitivas que se manifestaram em uma proporção superior ao percentual mínimo estipulado em relação ao total de quesitos avaliados.

## 4.2 Constante `DEFAULT_LIMIARES`

python

```
DEFAULT_LIMIARES = {    "transversal_min_frequencia": 3,<br/>    "transversal_min_quesit
```

## 4.3 Funções do Módulo

`_carregar_limiares(base_cfg: Path | None) -> dict`

Efetua a leitura dos parâmetros de corte pedagógico do sistema a partir de arquivo de configuração em disco. Caso o caminho não seja informado ou ocorra falha de I/O, retorna a estrutura `DEFAULT_LIMIARES`.

`_carregar_nomes_criterios(base_cfg: Path | None, faixa: str) -> dict`

Extraí a descrição nominal e os códigos técnicos dos critérios a partir da matriz da faixa em `config/faixas/{faixa}.json`. Caso o arquivo inexista, utiliza a tabela `CRITERIOS_FALLBACK`.

`_nome_criterio(nomes: dict, quesito: str, indice: int) -> str`

Recupera a descrição textual do critério utilizando indexação baseada em 1 (1-based), preservando a compatibilidade com a numeração física das folhas de exame.

`_frequencias_por_criterio(avaliacoes: dict) -> dict`

Normaliza os dados heterogêneos de entrada para o formato vetorial matricial: `{quesito: {indice: freq}}`.

`_regra_transversal(freqs: dict, nomes: dict, lim: dict) -> list[dict]`

Executa a varredura e monta a lista de observações qualificadas sob a regra transversal.

`_regra_concentrada(freqs: dict, nomes: dict, lim: dict) -> list[dict]`

Aplica os limiares de concentração e isola falhas agudas de execução.

`_regra_ranking(freqs: dict, nomes: dict, lim: dict) -> list[dict]`

Computa as somas absolutas por critério e gera a observação de maior impacto negativo.

`_regra_percentual(freqs: dict, nomes: dict, lim: dict) -> list[dict]`

Avalia a cobertura percentual dos erros em relação ao espectro de quesitos.

```
gerar_observacoes_automaticas(avaliacoes: dict, faixa: str = "branca",
base_cfg: Path | None = None) -> list[dict]
```

Aplica consecutivamente as 4 regras de inferência lógica sobre a matriz normalizada de frequências, ordena as mensagens por ordem de relevância técnica e trunca a lista resultante de acordo com o parâmetro `max_observacoes`.

```
merge_no_json(resultado: dict, base_cfg: Path | None = None) -> dict
```

Lê a propriedade `aluno.faixa_atual` dentro da estrutura de resultados consolidados de um aluno, executa a geração de observações e persiste a lista sob a chave `observacoes_automaticas` no próprio dicionário.

## 5. Utilitários de Nomes ( `core/nomes.py` )

O módulo `core/nomes.py` é dedicado estritamente ao tratamento de strings de identificação dos candidatos para fins de diagramação visual.

```
abreviar_nome(nome: str, max_len: int = 22) -> str
```

Formata o nome completo de um atleta para o padrão: primeiro nome + inicial do primeiro sobrenome foneticamente relevante + último sobrenome integral (exemplo: "Pedro José da Silva" converte-se para "Pedro J. Silva"). Conectivos intermediários ("da", "de", "do", "das", "dos") são ignorados no cômputo da inicial.

*Nota de Implementação: A função `abreviar_nome` deve ser invocada exclusivamente na geração física das folhas pelo `pre_exame.py` para acomodar o texto na caixa fixa da prancheta. Relatórios HTML (Sensei e Master) devem sempre utilizar a string completa do nome do atleta.*

## 6. Geradores de Relatórios HTML ( `core/relatorio_html.py` )

O módulo `core/relatorio_html.py` constrói documentos HTML autossuficientes, encapsulando regras CSS internas projetadas para compatibilidade dupla: visualização fluida em tela e impressão fiel em papel tamanho A4 (210 × 297 mm).

### 6.1 Tabelas de Estilos e Constantes Visuais

- `COR_STATUS` : Mapeamento de paleta cromática hexadecimal para marcação visual de status ( `APROVADO` , `APROVADO_PONTO_ATENCAO` , `RECUPERACAO` , `REPROVADO` , `REVISAO_PENDENTE` , `AUSENTE` ).
- `COR_QUESITO` : Paleta de identificação temática para os quatro quesitos fundamentais nos gráficos de dispersão e barras.

### 6.2 Métodos Principais de Interface

```
gerar_html_exame(resultados: list, regras: dict, recomendacoes: dict,
dojo_id: str = "DOJO", exame_id: str = "", titulo: str = "Relatório do
```

```
Sensei", nomes: dict = None, sensei_responsavel: str = "", avaliadores_map: dict = None) -> str
```

Compila a interface visual completa do Relatório do Sensei. Constrói cards individuais de atletas, renderiza as mini-barras horizontais de aproveitamento por quesito, projeta a tabela de frequências cruzadas com nomes nominais dos avaliadores oficiais, categoriza observações pontuais ("BOM!" e "A MELHORAR") com registro explícito de autoria e anexa o ranking com a indicação da coluna "Nova Faixa" (derivada da ordem oficial de graduação).

```
salvar_html_exame(conteudo_html: str, saida_caminho: Path) -> Path
```

Persiste a string do relatório do sensei em disco no caminho designado, instanciando programaticamente os diretórios pais intermediários caso não existam no sistema de arquivos.

```
gerar_html_master(resultados_dojos: dict, regras: dict, recomendacoes: dict, titulo: str = "Relatório Master Consolidado", exame_id: str = "", nomes: dict = None, avaliadores_map: dict = None) -> str
```

Gera o documento analítico global para a diretoria técnica. Agrega e sintetiza os indicadores de todos os dojos participantes, tabula a aderência e médias percentuais por quesito, destaca os alunos em zona de atenção, expõe a matriz de critérios mais penalizados em todo o evento e injeta os textos contextualizados de "Recomendação:" e "Planejamento Sugerido:" a partir de `config/recomendacoes.json["por_quesito"]`.

```
salvar_html_master(conteudo_html: str, saida_caminho: Path) -> Path
```

Persiste o artefato compilado do Relatório Master na estrutura de armazenamento de saída.

### 6.3 Métodos Utilitários Internos

O módulo decompõe a renderização em sub-rotinas utilitárias dedicadas: `_esc` (sanitização de caracteres especiais HTML), `_status_cor`, `_carregar_avaliadores_map`, `_carregar_ordem_faixas`, `_nova_faixa`, `_classificar_obs`, `_separar_obs`, `_texto_recomendacao`, `_texto_recomendacao_detalhe`, `_tabela_marcacoes`, `_obs_com_autoria`, `_card_aluno_sensei`, `_bloco_notas_quesito_sensei`, `_agregar_criterios`, `_agregar_recomendacoes`, `_bloco_marcacoes_quesito`, `_bloco_recomendacoes_master`, `_bloco_analise_desempenho`, `_bloco_notas_quesito` e `_bloco_observacoes_master`.

## 7. Orquestração e Pipeline de Execução ( `core/pipeline.py` )

O arquivo `core/pipeline.py` consolida as chamadas de ponta a ponta e gerencia a transferência controlada de dados entre o processamento em memória e o sistema de arquivos local e em nuvem.

## 7.1 Interface de Linha de Comando (CLI)

bash

```
python core/pipeline.py --config config --data data --output output [--pasta-omr output/
```

- `--config` : Caminho para o diretório de arquivos de configuração (default: `'config'` ).
- `--data` : Diretório contendo os bancos de dados cadastrais e exames (default: `'data'` ).
- `--output` : Diretório base para emissão de artefatos (default: `'output'` ).
- `--pasta-omr` : Localização dos arquivos JSON gerados pelo OMR (default: `<output>/json` ).
- `--no-drive` : Flag booleana (`store_true`). Quando informada, suprime qualquer tentativa de upload local para o drive montado, delegando a sincronização exclusivamente ao agente de CI via rclone.

## 7.2 Funções de Fluxo

```
gerar_relatorios_html(resultados: list, regras: dict, recomendacoes: dict,  
exame_id: str, dojo_id: str, rel_dir: Path, nomes: dict = None,  
sensei_responsavel: str = "", avaliadores_map: dict = None) -> list[Path]
```

Compila os dados de execução e grava em disco os artefatos `relatorio_sensei_{EXAME}_{DOJO}.html` e `relatorio_master_{EXAME}.html` . Mantém estritamente o padrão de nomenclatura indexado por exame para evitar sobrescritas durante avaliações múltiplas no mesmo ciclo anual.

```
_criar_staging_distribuicao(rel_dir: Path, exame_id: str, dojo_id: str,  
sensei_responsavel: str) -> Path
```

Organiza a estrutura de pastas temporárias em `output/distribuicao/` replicando rigorosamente o esquema de permissões do Google Drive. Deriva os nomes estáveis de diretórios a partir de `config/dojos.json` , populando:

text

```
output/distribuicao/├─ master/{EXAME}/└─ senseis/{Sensei}/{EXAME}/
```

```
_enviar_staging_para_drive(staging: Path, rel_dir: Path) -> bool
```

Executa a transferência física dos arquivos presentes na árvore de staging para a unidade de rede ou drive montado (G:) quando o script opera em modo convencional (sem a flag `--no-drive` ).

```
main(argv: list = None) -> int
```

Ponto de entrada do executável. Carrega tabelas de regras, lê os arquivos intermediários gerados pelo OMR, extrai metadados do evento, instancia tabelas relacionais de nomes e senseis, processa a pontuação de todos os alunos, gera a documentação HTML, monta a área de staging e retorna o código de terminação 0 em caso de êxito integral.

## 8. Ferramentas Auxiliares de Apoio ( tools/ )

### 8.1 tools/pre\_exam.py

Gera os documentos imprimíveis em formato PDF contendo os balões de marcação e as grades de observação técnica ("BOM!" e "A MELHORAR"), acompanhados dos respectivos códigos QR de identificação serial. Produz simultaneamente os arquivos \*\_coordenadas.json necessários para calibrar os offsets do scanner.

- Argumentos suportados: --exame (obrigatório), --csv , --aluno (parâmetro cumulativo append), --avaliador (cumulativo append), --dojo , --saida (default: output/pre\_exam ) e --validar .

### 8.2 tools/ingest\_folhas.py

Responsável pela leitura óptica das imagens digitalizadas, detecção de marcadores fiduciais e interpolação de preenchimento dos balões.

- Argumentos suportados: --entrada , --config , --saida , --arquivo , --origem (aceita os modos scanner ou auto ).
- Código de terminação: Retorna status de saída 3 caso nenhuma imagem válida seja encontrada no diretório de entrada, permitindo ao pipeline de automação interromper o fluxo sem registrar estado de falha fatal.
- Gestão de persistência: Move as imagens devidamente processadas para o diretório indicado pelo parâmetro --arquivo , assegurando a idempotência do fluxo diário.

## 9. Diretrizes Transversais de Engenharia e Qualidade

| Diretriz                            | Escopo                  | Impacto de Não Conformidade                                                                      |
|-------------------------------------|-------------------------|--------------------------------------------------------------------------------------------------|
| Trava Ética de Controle             | core/engine.py          | Violação pedagógica crítica e perda de homologação federativa da avaliação técnica.              |
| Migrações por Identificador Nominal | Banco de dados / Config | Corrupção das séries históricas de notas decorrente de alteração na ordem das colunas da matriz. |
| Fixtures de Teste Não Sintéticas    | tests/                  | Geração de falsos-positivos na esteira de integração contínua (CI).                              |

| Diretriz                           | Escopo         | Impacto de Não Conformidade                                                                               |
|------------------------------------|----------------|-----------------------------------------------------------------------------------------------------------|
| Arredondamento<br>Decimal Prévio   | core/engine.py | Classificação indevida de alunos localizados na fronteira matemática entre APROVADO e RECUPERAÇÃO.        |
| Tratamento<br>Diferencial de Nomes | core/nomes.py  | Truncamento indevido em relatórios de impressão física ou estouro de layout nas pranchetas de arbitragem. |

A integridade da Trava Ética de Falta de Controle (Fase 2 do playbook técnico) constitui um requisito não-negociável da arquitetura do Karate-Ashi: qualquer refatoração nas classes matemáticas deve demonstrar conformidade através de testes unitários que comprovem a rejeição sumária de notas cheias em cenários de descontrole físico de golpes.

Todas as transformações em esquemas de dados devem ser conduzidas utilizando o nome textual do critério como chave primária de correlação, jamais a posição relativa ou índice numérico da folha. Os testes de integração automatizados devem obrigatoriamente carregar as matrizes reais contidas em `config/` para evitar derivações funcionais silenciosas.

O arredondamento da nota final composto pela soma dos 4 quesitos deve ser processado a uma casa decimal utilizando o método `ROUND_HALF_UP` da biblioteca Decimal antes da avaliação lógica do status do candidato, eliminando divergências de precisão binária de ponto flutuante.

Por fim, o emprego de `abreviar_nome` é restrito à fase de pré-impressão de formulários físicos para manter legibilidade visual na área útil do papel A4; os relatórios de gestão devem, invariavelmente, preservar os registros nominais integrais dos praticantes.

---

*Documento gerado a partir dos fontes do repositório karateashi no branch main — para manutenção e evolução do sistema Karate-Ashi.\**